Agentic SDLC Documentation & Design Platform

CrewAI Multi-Agent Architecture & Responsibilities

Design proposal | Version 1.0 | 7 November 2026

Status: proposed architecture; not implemented or tested. Companion to Agentic_SDLC_Project_Specification.docx.

1. Design Goal and Boundaries

Convert client assignments (PDF, DOCX, TXT) into source-grounded SDLC first drafts: requirements, assumptions, architecture, entity model, ER diagram, workflow/sequence diagrams, API design when relevant, test plan/protocol, test-case suite specification, traceability, and deployment/readiness recommendations. The platform does not independently approve business policy, certify designs, run generated tests, or claim production readiness.

2. Architecture Overview

Browser / Gradio UI → Upload & Parsing Service → Job Orchestrator (CrewAI) → Specialized agents → Structured Artifact Store → Consistency Gate → Human Review → Document/Diagram Renderer → Download (DOCX, diagram source, ZIP).

All agents read the same versioned project context and return Pydantic-validated structured data. The orchestrator passes approved requirements to design agents and design outputs to QA. Deterministic services handle file parsing, rendering, packaging and storage; they are not LLM agents.

2.1 Execution strategy

- Use sequential tasks with explicit dependencies for the MVP. A manager coordinates tasks and reruns; avoid unrestricted agent-to-agent delegation.
- Human checkpoint A: confirm requirements and assumptions before generating downstream designs. Human checkpoint B: review consistency warnings and approve exports.
- Later, add parallel independent diagram/test drafting only after deterministic merge and validation are reliable.

3. Crew and Agent Responsibilities

Component	Primary responsibility	Key output
Manager / Orchestrator	Plan task order, pass shared context, enforce checkpoints, handle retries and errors; no autonomous business approval.	Job state, task statuses, manifest
Intake & Requirements Agent	Extract actors, requirements, constraints, acceptance criteria and ambiguities; cite uploaded source.	Requirements register, assumptions, open questions

Architecture & Domain Agent	Propose solution boundaries, components, entities, relationships, data dictionary, alternatives and trade-offs.	Architecture model, entity/ER model
Workflow & API Agent	Describe use cases, interactions, sequence steps, API contracts and error cases where relevant.	Sequence models, endpoint specification
QA & Test Design Agent	Derive test strategy, protocol, positive/negative/boundary cases, and requirement-to-test mappings.	Test plan, test-case suite specification, traceability
Consistency & Review Agent	Cross-check all artifacts for unsupported claims, mismatched entity names, missing requirements and coverage gaps.	Validation findings, severity, suggested fixes
Document Publisher (service)	Render approved structured artifacts to Word, diagram source/visuals, manifest and ZIP.	Downloadable artifact package

3.1 Agent contracts

Intake & Requirements Agent

- Input: Parsed source text with page/section references
- Output: Requirement {id, statement, type, source_ref, status}; assumption; open_question
- Guardrail: Never convert a guess into a source requirement.

Architecture & Domain Agent

- Input: Approved requirement IDs, constraints, approved assumptions
- Output: Components, entities, attributes, relationships, decisions, alternatives
- Guardrail: Mark all unapproved design choices as proposals.

Workflow & API Agent

- Input: Requirements, domain model, selected use cases
- Output: Sequence participants/messages; API operations, DTOs, status/error scenarios
- Guardrail: Do not invent an API if the assignment calls for another interface.

QA & Test Design Agent

- Input: Approved requirements, designs and API/workflow contracts
- Output: Test cases {id, requirement_ids, preconditions, steps, expected_result, type}; protocol
- Guardrail: Produce specifications, not fabricated executed test results.

Consistency & Review Agent

- Input: All artifact JSON plus source references
- Output: Findings {id, severity, artifact, related_ids, explanation, recommendation}
- Guardrail: Report gaps; do not silently overwrite approved requirements.

4. Shared Data Model and Traceability

Entity	Core fields / relationship
Project	project_id, title, source_file, created_at, status, revision

SourceSegment	segment_id, project_id, document/page/section, text_hash
Requirement	req_id, project_id, statement, kind, source_segment_ids, status
Assumption / Question	item_id, related_req_ids, text, proposed_by, resolution, approver
DesignArtifact	artifact_id, type, version, requirement_ids, structured_payload, review_status
TestCase	test_id, requirement_ids, steps, expected_result, category
ReviewFinding	finding_id, artifact_ids, severity, issue, resolution_status
GenerationRun	run_id, project_id, model_version, task_state, timings, errors

Traceability rule: each core requirement has a stable ID, and related entities, endpoints, sequences and tests reference those IDs. Export a requirements-to-artifacts-to-tests matrix. Preserve source text and approval status through regeneration.

5. Main Processing Sequence

- 1. User uploads a supported assignment. File validation and parsing extract text plus page/section anchors.
- 2. Requirements Agent creates source-linked requirements, assumptions and questions. Schema validation rejects malformed output.
- 3. User reviews and confirms the requirements baseline; unresolved questions remain visibly open.
- 4. Architecture & Domain Agent proposes components and ER model from that baseline.
- 5. Workflow & API Agent proposes sequence models and interfaces consistent with the domain model.
- 6. QA Agent produces test protocol, test cases and traceability links.
- 7. Consistency Agent checks completeness, contradictions and unsupported assumptions.
- 8. User reviews findings, edits or approves; affected tasks regenerate with versioned inputs.
- 9. Publisher renders DOCX, editable diagram source (Mermaid/PlantUML), optional diagram images and a ZIP manifest.

6. Diagram Generation Design

ER diagrams: generate a structured list of entities, fields, primary/foreign keys and cardinalities, validate references, then render Mermaid ER or PlantUML. Sequence diagrams: generate participants and ordered messages from use cases, validate participant names and step references, then render Mermaid/PlantUML. Store both editable source and rendered images; a render failure is a visible error, not a successful artifact.

7. Deliverables by Stage

Stage	Generated artifacts	Review condition
Requirements	Requirements specification, assumptions/open questions, use-case list	Source references and human confirmation
Design	Architecture document, entity/data dictionary, ER diagram, sequence diagrams, API specification	Consistency with approved requirements
Verification planning	Test strategy, test protocol, test suite specification, traceability	Coverage and expected-result checks

	matrix	
Operations proposal	Local run guidance, deployment/security/readiness checklist	Clearly labeled proposals
Export	Individual DOCX, diagram source/images, ZIP and manifest	Approved or explicitly draft status

8. CrewAI Implementation Sketch

Python modules: app/ui.py; app/ingestion.py; app/crew/agents.py; app/crew/tasks.py; app/crew/flow.py; app/schemas/*.py; app/validation.py; app/exporters/docx_export.py; app/exporters/diagram_export.py; app/storage.py; tests/.

Configure agent roles and task prompts in YAML or Python; define structured Pydantic output for each task. Use a manager-controlled sequential crew (or CrewAI Flow for explicit human checkpoints). Tool access is limited: source reader, schema validator, diagram renderer and document exporter. The manager never writes directly to the approved artifact store without validation.

8.1 Failure and retry policy

- Parsing failures stop the job with an actionable error.
- LLM output schema failures retry a limited number of times; preserve prior approved artifacts.
- Source contradictions produce review findings rather than invented resolutions.
- Task retries are idempotent by project, input version, task name and generation configuration.

9. Security and Governance

- Treat uploads as untrusted content and ignore instructions embedded in source documents that attempt to redirect agents.
- Enforce file type/size limits, malware scanning where available, access controls, and project-level isolation.
- Do not send confidential client documents to unapproved LLM providers; establish retention/deletion rules.
- Log task metadata, timings and errors; redact source content and sensitive data from diagnostics.
- Keep human approval and generated-versus-verified status explicit in every document.

10. MVP Scope, Acceptance and Tests

First demo: three agents (Requirements, Design, QA/Validation), one orchestration flow, Gradio upload/review/download, three Word outputs, one ER diagram, one sequence diagram and ZIP export. Workflow/API and dedicated Consistency agents are planned extensions; their responsibilities may initially be implemented as structured tasks within the three-agent MVP.

- Upload two different, nonconfidential assignments; verify correct extraction and source references.
- Confirm requirements and assumptions are separate and approvals gate downstream generation.
- Check generated DOCX opens; render both diagram sources without errors.
- Verify test cases reference real requirement IDs and an injected mismatch is flagged.
- Confirm downloadable ZIP includes manifest and correct version/status labels.
- Record generation time and manual corrections; do not claim unmeasured time savings.

10.1 Suggested test commands (from repository root, once implemented)

```
python -m pytest -q
python app/ui.py  # Gradio launch entrypoint, after implementation
```

The commands above describe a proposed repository. The tests and entrypoint must be created before they can run.

11. Future Extensions

Add OpenAPI generation, code scaffolding, executable test generation, GitHub/Jira integration, approval versioning, incremental regeneration, multi-tenant hosting and agent quality evaluation. These are explicitly outside the initial demonstration.